

Regression Failure Bucketing

Grant Gung-Yu Pan

Synopsys, Inc.

0. Revision History

2026-06-01	-	Update runtime limit and renew benchmarks to avoid diff-bugs-same-syndrome
2026-05-21	-	Add 3.6. sample programs
2026-05-16	-	Modify sim and trace logs into .gz format
2026-02-12	-	Add explanation for Ibex cases and refine some ambiguous descriptions
2026-01-22	-	Initial version

1. Introduction

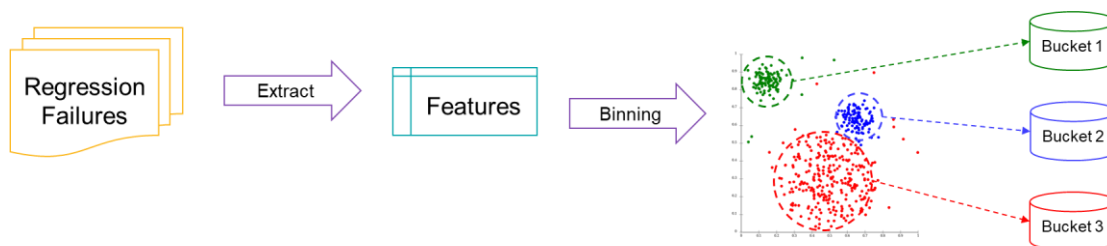


Fig. 1: Regression failure bucketing flow.

RTL regression debugging often involves thousands of failing simulations across different tests, seeds, and configurations. Analyzing these failures individually is inefficient; instead, engineers group failures into buckets that correspond to distinct root causes (bugs).

Traditional rule-based bucketing relies on handcrafted heuristics that are hard to scale and maintain, especially when failures depend on complex combinations of signals, instructions, or runtime conditions. In contrast, data-driven bucketing adapts automatically to new failure patterns and scales better with large volumes of semi-structured log data.

In this problem, contestants are asked to automatically bucket regression failures using simulation logs generated from a modified Ibex RISC-V core. The conceptual flow is shown in Fig. 1, including feature extraction and binning (clustering).

2. Problem Formulation and Input/Output Format

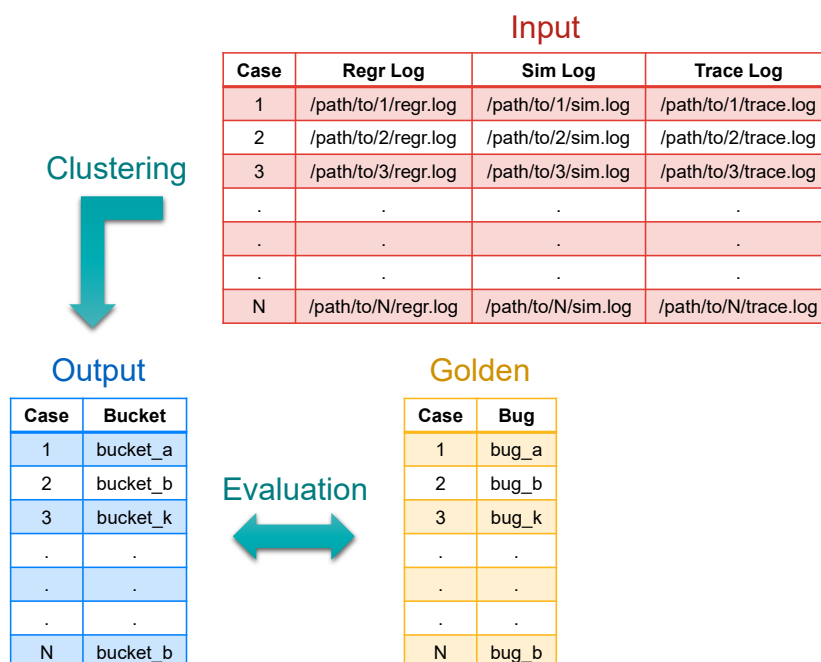


Fig. 2: Problem formulation: input table and output table.

The benchmarks are generated using Ibex, which is a production-quality open source 32-bit RISC-V CPU core written in SystemVerilog. Please refer to the public documents for detailed information:

- <https://ibex-core.readthedocs.io/en/latest/>
- <https://github.com/lowRISC/ibex>

For this problem, the source codes are modified to contain some bugs and then sent to simulation. To mimic the scenario of daily regression debugging, the passing cases are omitted, while only the failure cases are sent for bucketing.

As illustrated in Fig. 2, given N failure cases simulated by different tests and random seeds, the goal is to cluster into K buckets for further debugging. Each golden bucket corresponds to one injected bug. All failure cases caused primarily by the same bug belong to the same golden bucket, even if the observable symptoms vary. One injected bug can cause multiple failure cases across different tests and seeds. However, each failure case is assumed to be caused by one injected bug, so each row maps to exactly one golden bucket.

2.1. Format of Input Data

There is an input table in CSV format containing the paths for files. The table contains N rows from case 1 to case N, a Case identifier column plus three columns for log file paths belonging to the case.

The logs contain recurring text patterns, such as mismatch lines, UVM_FATAL messages, PC traces. Clustering relies on pattern similarity, so it is vital to extract proper features to generate better bucketing results.

2.1.1. regr.log

This is the snippet of overall regression report, cutting for the failure case. In the Ibex DV flow, tests run the RTL and a reference ISS (Spike) in co-simulation, and a checker compares architectural effects instruction-by-instruction. A “Mismatch” means the RTL’s instruction state (PC, registers, memory effects) diverged from the ISS, and a FAILED line is the testbench verdict.

Here is the example regr.log of Case-A containing mismatching of instruction(s).

```
Mismatch[1]:
ibex[75] : pc[800001d6] c.lui    x25,0xd: s9:0000d000
spike[75] : pc[ffffffff8000020c] lui    gp, 0xbf802: gp:bf802000
[FAILED]: 77 matched, 4262 mismatch
```

Mismatch[1]: lines enumerating differences (e.g., PCs/registers) are typical checker outputs when RTL≠ISS. The snippet indicates the first mismatch occurred at retire index 75; the RTL retired a compressed LUI at PC 0x800001d6, while Spike expected a different instruction and PC. The final counter shows many subsequent divergences after the first break in lockstep. This is exactly the kind of text symptom the DV co-sim scoreboard emits when traces diverge.

Here is the example regr.log of Case-B containing only the failure test name.

```
riscv_debug_basic_test.0 : [FAILED]
```

In contrast, a short line like riscv_debug_basic_test.0 : [FAILED] means the test declared failure without the checker dumping detailed mismatch context, for example, due to a separate UVM fatal/timeout/termination.

For the details, please refer to this Ibex documentation:

- https://ibex-core.readthedocs.io/en/latest/03_reference/verification.html

2.1.2. sim.log.gz

This is the simulation reported generated by Synopsys VCS. It contains information from the testbench, especially UVM lines. This file is the raw UVM console log from the environment that runs Ibex DV tests. It is expected to include the VCS header, UVM report macros: UVM_INFO, UVM_WARNING, UVM_ERROR, UVM_FATAL.

Here is the example sim.log.gz of Case-A containing only UVM_INFO.

```
Command: ibex/dv/uvm/out/rtl_sim/vcs_simv +vcs+lic+wait +signature_addr=0x8ffffffc -ucli -do ibex/dv/uvm/vcs.tcl +ntb_random_seed=7
+UVM_TESTNAME=core_ibex_base_test +bin=ibex/dv/uvm/out/instr_gen/asm_tests/riscv_machine_mode_rand_test_0.bin -l sim.log
+UVM_VERBOSITY=UVM_HIGH
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version U-2023.03-SP1_Full64; Runtime version U-2023.03-SP1_Full64; Mar  8 06:05 2025
UVM_INFO /global/apps/vcs_2023.03-SP1/etc/uvm-1.2/base/uvm_root.svh(402) @ 0: reporter [UVM/RELNOTES].
.
.
.
UVM_INFO /global/apps/vcs_2023.03-SP1/etc/uvm-1.2/base/uvm_objection.svh(1276) @ 140617000: reporter [TEST_DONE] 'run' phase is ready to
proceed to the 'extract' phase

--- RISC-V UVM TEST FAILED ---
.
.
.
```

A transcript full of UVM_INFO and no UVM_ERROR/FATAL/WARNING but ending with --- RISC-V UVM TEST FAILED --- indicates the test's functional checker or end-of-test condition decided it failed due to mismatches or an unmet condition, though the UVM infrastructure didn't raise a fatal error itself.

Here is the example sim.log.gz of Case-B containing UVM_FATAL.

```
Command: ibex/dv/uvm/out/rtl_sim/vcs_simv +vcs+lic+wait +signature_addr=0x8ffffffc -ucli -do ibex/dv/uvm/vcs.tcl +ntb_random_seed=7
+require_signature_addr=1 +max_interval=500 +enable_debug_seq=1 +UVM_TESTNAME=core_ibex_debug_intr_basic_test
+bin=ibex/dv/uvm/out/instr_gen/asm_tests/riscv_debug_basic_test_0.bin -l sim.log +UVM_VERBOSITY=UVM_HIGH
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version U-2023.03-SP1_Full64; Runtime version U-2023.03-SP1_Full64; Mar  8 06:09 2025
UVM_INFO /global/apps/vcs_2023.03-SP1/etc/uvm-1.2/base/uvm_root.svh(402) @ 0: reporter [UVM/RELNOTES]
.
.
.
UVM_FATAL ibex/dv/uvm/tests/core_ibex_test_lib.sv(335) @ 93625000: uvm_test_top [uvm_test_top] No dret detected, or incorrect privilege
mode switch in timeout period of 20000 cycles

--- RISC-V UVM TEST FAILED ---
.
.
.
```

Here the testbench raised a UVM_FATAL because a debug-mode return (dret) wasn't observed or privilege mode didn't switch as expected within a timeout.

For the details, please refer to this UVM documentation:

- <https://www.chipverify.com/uvm/report-functions>

2.1.3. trace.log.gz

This is the CPU trace log generated by the `ibex_tracer`, this is a tab-separated per-instruction retire log with columns: Time | Cycle | PC | Insn | Decoded instruction | Register/memory contents.

Here is the example trace.log.gz of Case-A.

Time	Cycle	PC	Insn	Decoded instruction	Register and memory contents			
	253000		26	80000080	00000013	addi	x0,x0,0	x0:0x00000000 x0=0x00000000
	255000		27	80000084	4081	c.li	x1,0	x1=0x00000000
	257000		28	80000086	5296a137	lui	x2,0x5296a	x2=0x5296a000
	279000		39	8000008a	ec010113	addi	x2,x2,-320	x2:0x5296a000 x2=0x52969ec0
	281000		40	8000008e	4181	c.li	x3,0	x3=0x00000000
.								
.								
.								
	140505000		70152	800086f6	903f2723	sw	x3,-1778(x30)	x30:0x8000c6f2 x3:0x00000001 PA:0x8000c000 store:0x00000001
load:0x00000001								
	140509000		70154	800086fa	bfe5	c.j	800086f2	
	140539000		70169	800086f2	00004f17	auipc	x30,0x4	x30=0x8000c6f2
	140589000		70194	800086f6	903f2723	sw	x3,-1778(x30)	x30:0x8000c6f2 x3:0x00000001 PA:0x8000c000 store:0x00000001
load:0x00000001								
	140593000		70196	800086fa	bfe5	c.j	800086f2	

One row indicates at time=140,505,000, cycle=70152, PC=0x800086f6, the retired instruction is a store; the tracer shows the base register value (x30), source register (x3), the physical address (PA) that was accessed, and the data written/read. This aligns with the tracer's "register and memory contents" description.

Here is the example trace.log.gz of Case-B.

Time	Cycle	PC	Insn	Decoded instruction Register and memory contents					
	229000		14	80000080	00000013	addi	x0,x0,0	x0:0x00000000	x0=0x00000000
	251000		25	80000084	4081	c.li	x1,0	x1=0x00000000	
	253000		26	80000086	5296a137	lui	x2,0x5296a	x2=0x5296a000	
	275000		37	8000008a	ec010113	addi	x2,x2,-320	x2:0x5296a000	x2=0x52969ec0
	277000		38	8000008e	4181	c.li	x3,0	x3=0x00000000	
	.								
	.								
	.								
	93567000		46683	80004f9e	8e6d	c.and	x12,x11	x12:0xe7c0009d	x11:0xffffffffbe x12=0xe7c0009c
	93571000		46685	80004fa0	034d7fb3	remu	x31,x26,x20	x26:0x8001ce7c	x20:0x00000000 x31=0x8001ce7c
	93601000		46700	80004fa4	4034dcb3	sra	x25,x9,x3	x9:0x00000042	x3:0x7ea5e000 x25=0x00000042
	93615000		46707	80004fa8	03af9633	mulh	x12,x31,x26	x31:0x8001ce7c	x26:0x8001ce7c x12=0x3ffe3187
	93617000		46708	80004fac	018b1693	slli	x13,x22,0x18	x22:0xffffffff	x13=0xff000000

The trace shows normal execution progressing through arithmetic and bit-manipulation instructions. The PC advances monotonically, and register updates look valid, indicating that the core is still retiring instructions normally.

For the details, please refer to this Ibex tracer documentation:

- https://ibex-core.readthedocs.io/en/latest/03_reference/tracer.html

2.1.4. Case Analysis

The failure examples, Case-A and Case-B, come from different bugs, so they should be clustered into different buckets.

Case-A fails because of the instruction mismatches between RTL and ISS shown in `regr.log`, so there is no `UVM_ERROR` or `UVM_FATAL` in `sim.log.gz`, and there are some repeating patterns in `trace.log.gz` tail lines.

On the other hand, Case-B fails because of `UVM_FATAL` in `sim.log.gz`, so there is no mismatching result in `regr.log` and no weird behavior in `trace.log.gz` until it is shut down suddenly.

2.2. Format of Output Data

The output table in CSV format contains N rows, as the same as the input table. The table should contain two columns: Case (integer case ID) and bucket (bucket label) for each case. The bucket label can be any string representing the categories. Cases with the same bucket label belong to the same bucket.

2.3. Objective

The goal is to produce the correct bucketing result under a timing constraint. Each golden bucket corresponds to one injected bug in the design. Because multiple valid groups may exist, correctness is measured only by the evaluation metric described in Section 3, not by exact matching of bucket labels.

All failure cases caused primarily by the same bug belong to the same golden bucket, even if the observable failure symptoms differ.

Using Large Language Model (LLM) is acceptable, while the runtime needs to be considered. See the Evaluation Section for details.

3. Evaluation

3.1. Figure-of-Merit

Treat each pair (i, j) as positive if they're in the same golden bucket, otherwise negative.

True Positive Rate (TPR) = $TP / P = TP / (TP + FN)$

True Negative Rate (TNR) = $TN / N = TN / (TN + FP)$

Balanced Accuracy = $(TPR + TNR) / 2$

Notice the given K is only a soft hint for the number of buckets. Using the pairwise balanced accuracy, the results can still get a score even if the count is not exactly k. Balanced accuracy is computed on whether pairs of cases are grouped correctly. Therefore, if one bucket is accidentally split (or two small buckets merged), most pairwise relationships may still be correct, yielding a high score.

3.2. Benchmark

Ten benchmarks will be used to evaluate. The Balanced Accuracy of the ten benchmarks are averaged to be the final score. If any benchmark fails to run successfully within runtime limit, the score of this benchmark is 0.

Benchmark	Max Lines	Max Cases	Buckets	Max Runtime	Public
1	1M	10	2	30 s	Yes
2	1M	30	4	30 s	Yes
3	10M	100	8	100 s	No
4	10M	300	16	100 s	No
5	10M	1000	32	100 s	No
6	10M	3000	64	100 s	No
7	100M	100	8	300 s	No
8	100M	300	16	300 s	No
9	100M	1000	32	300 s	No
10	100M	3000	64	300 s	No

PS. Runtime limit will be tripled and doubled in Alpha and Beta testing, respectively.

3.3. Problem Interface

The submitted program is as follows:

```
%> regr_fail_bucketing --input <input.csv> --output <output.csv> --k <k>
```

For the program written in Python, it is strongly recommended to pack the entire environment using pyinstaller. Uploading the source code is optional; only if the executable program cannot run, the evaluator will try running the source code instead.

3.4. LLM Usage & Configuration

Using LLM endpoint is optional where the token cost is not counted. While encouraged, contestants need to handle the side effects, such as hallucinations, non-determinism, runtime overhead, etc. It is strongly recommended to implement exception handling logics inside the program when accessing LLM endpoints. If the program needs to accept an environment variable `LLM_MODEL_CONFIG`. Inside the program, the config needs to be loaded by:

```
yaml.safe_load(os.getenv("LLM_MODEL_CONFIG"))
```

The environment variable will be pointed to the two evaluation endpoints shown below.

```
# oss.yaml
completion:
  model_name: "qwen3-coder-480b-a35b-inst-fp8"
  config:
    api_key: "<API-KEY>"
    base_url: "<BASE-URL>"
    client_library: "openai.AsyncOpenAI"
    model: "<MODEL-PROVIDER>"
embedding:
  model_name: "nomic-embed-text-v1.5"
  config:
    api_key: "<API-KEY>"
    base_url: "<BASE-URL>"
    client_library: "openai.AsyncOpenAI"
    model: "<MODEL-PROVIDER>"
```

```
# openai.yaml
completion:
  model_name: "gpt-4o"
  config:
    api_key: "<API-KEY>"
    base_url: "<BASE-URL>"
    client_library: "openai.AsyncOpenAI"
    model: "<MODEL-PROVIDER>"
embedding:
  model_name: "text-embedding-3-small"
  config:
    api_key: "<API-KEY>"
    base_url: "<BASE-URL>"
    client_library: "openai.AsyncOpenAI"
    model: "<MODEL-PROVIDER>"
```

The evaluation is run once for each of the two endpoints and the higher score is used. During development, contestants could prepare their own config to connect to public or own private LLM endpoints. Here is the overall policy for using LLM:

- Contestants may use LLM APIs for log understanding or embedding.
- Network latency is included in the runtime limit.
- The evaluation will be executed once per benchmark.
- Contestants are responsible for handling non-determinism.
- Excessive API calls that cause timeouts will lead to zero score for that benchmark.

3.5. Handling Randomness

The evaluation is only run once for each benchmark, so it is strongly recommended to minimize uncertainty. Some common takeaways to reduce randomness in Python:

- Python
 - `random.seed(seed)`
 - `os.environ["PYTHONHASHSEED"] = str(seed)`
 - `np.random.seed(seed)`
- LLM
 - temperature
 - top_p

3.6. Sample Programs

Two example Python programs are provided:

- Without LLM: `char_embedding/regr_fail_bucketing`
- With LLM: `naive_completion/regr_fail_bucketing`

The reference programs are workable samples that follow the program interface in 3.3. However, those simple algorithms do not guarantee correct answers, even for small public benchmark sets. The samples are written in plain Python source for the ease of understanding; the contestants are free to submit any standalone executable.